assignment 4

Full Stack App

Design, Build, Ship · MPCS 51238 · Autumn 2026

Due Tuesday, October 27, at 5:30 PM

Overview

Over the first three weeks we built prototypes, now we'll go deeper into the stack: Clerk for auth, Supabase for data, and a public API for rich content. Extend Assignment 3 or start something new. The goal is for your classmates to be able to create accounts in your app and use it; each person sees their own data.

This assignment is also a learning journey. Each step ends with an Ask Your Agent prompt, so your agent teaches you what you just built: it explains it, draws it, or quizzes you on it.

Steps

1. Choose Your Tool and Your API

Extend and iterate on your Assignment 3 tool, or start something new. Add an API that makes it more useful, so someone can search or browse its data, save what they like, and come back to it.

Pick one of these free APIs, or find your own. All of these work without an API key:

- Weather: Open-Meteo
- Books: Open Library
- Music: iTunes Search
- Chicago bikes: Divvy
- City data: the Chicago Data Portal
- Art: the Art Institute of Chicago
- Recipes: TheMealDB
- Pokémon: PokéAPI

Think about the design process here: explore possibilities, so you don't end up making the same Pokémon app as your classmate.

Before you build on it, have your agent call it and show you one real response. What fields come back? What happens when the search finds nothing?

> ASK YOUR AGENT
>
> “Explain this response: what each field means, and which ones my tool needs.”

2. Plan the Data with Your Agent

Plan with your agent: discuss what gets saved, and to whom it belongs. Keep it small: most tools need two or three tables. Every table that holds someone's data has a `user_id` column. Write your data model in `AGENTS.md`, and read it before anything is built.

ASK YOUR AGENT

"Draw a diagram of my tables: what each one holds, and how each row belongs to one person."

3. Protect Your Keys

Your keys go in `.env.local`, never in your code or your commits. Before you paste in the first key, add a deny rule so your agent can't read the file, in `.claude/settings.json`:

```
{ "permissions": { "deny": ["Read(./.env)", "Read(./.env.*)"] } }
```

You'll create `.env.local` yourself, since the rule blocks the agent from writing it too.

ASK YOUR AGENT

"Explain why this deny rule protects my keys. Then explain the difference between my publishable and secret keys, and where each one is allowed to go."

4. Add Sign-In with Clerk

Add Clerk with `npx -y clerk@latest init`. Make sure a signed-out visitor can't reach anyone's data, and check that by trying it signed out.

ASK YOUR AGENT

"Show me how the Clerk CLI works: what `clerk init` added to my project, file by file."

5. Add Supabase and Connect It to Clerk

Create a Supabase project and connect its MCP server to your agent, scoped to that one project. Your agent does the setup work through the server; you should understand what it's doing. If you want your agent to only read your database, add `&read_only=true` to the server's URL, but for this project, it's recommended you let your agent write. Connect Clerk to Supabase with Supabase's third-party auth (Authentication → Sign In / Providers → Clerk). Have your agent create your tables as migrations, with row-level security on every table, so each person sees only their own rows.

ASK YOUR AGENT

"Explain each row-level security policy in plain words, then show me a query that one of them would block."

6. Where Should You Call the API?

Decide where the API call happens, in the browser or on your server, and be ready to say why: keys, rate limits, caching, or none of those. Design what someone sees while it loads, when it finds nothing, and when the API is down.

ASK YOUR AGENT

"Draw a diagram of one request, from the click to the screen: the browser, my server, Clerk, Supabase, and the API."

7. Test It, and Watch the Test Fail First

Write one end-to-end test with Playwright: two users sign in, each saves something, and neither can see the other's. Break your row-level security on purpose and watch the test fail, then fix it and watch it pass.

> ASK YOUR AGENT
>
> “Explain why the test failed when I broke row-level security, in one sentence I could say in class.”

8. Review and Deploy

Work on a branch. Before you merge, run `/security-review` and read your agent's changes yourself. Then deploy on Vercel with your environment variables set, and have a classmate make an account and use it.

> ASK YOUR AGENT
>
> “Quiz me on how my app works, one question at a time, and tell me what I got wrong. Start with the questions in Be Ready to Explain.”

Requirements

- **Accounts:** sign up, sign in, and sign out with Clerk; a signed-out visitor sees no one's data.
- **Data:** saved in Supabase, with row-level security on every table, so each person sees only their own.
- **API:** data from one external API, with designed states for loading, nothing found, and the API being down.
- **Persistence:** what you save is there after you sign out and back in, on another device.
- **Keys:** in environment variables, never in the code or the repository, with a deny rule for `.env` files.
- **Test:** one Playwright test that two users can't see each other's data, watched failing before it passes.
- **Review:** `/security-review` run on a branch, and its findings handled.
- **Deployed:** live on Vercel, where a classmate can make an account and use it.
- **UI/UX:** remember the design process, and the UI/UX design principles and flows we've discussed. You don't need to use everything we've learned, but keep it in mind as you build.

Tech Stack

Next.js and Tailwind CSS, Clerk for sign-in, Supabase for the database, and one external API. Put the code in your own GitHub repository and deploy it on Vercel's free Hobby plan. Stay on Clerk's development keys; production keys need a domain you own.

Your project's context file is `AGENTS.md`. If there's a `CLAUDE.md`, move anything useful into `AGENTS.md` and delete it, then check with `/context` that `AGENTS.md` loaded.

Submission

Submit in Google Classroom:

1. **Vercel URL:** a live link to your tool.
2. **GitHub URL:** your repository, with your `README.md` and a commit history that shows your progress.

Assessment

A complete assignment has:

- A live Vercel URL where a classmate can make an account and use the tool
- Sign-in with Clerk, and nothing visible signed out
- Data in Supabase with row-level security on every table
- Data from an external API, with loading, empty, and unavailable states
- What you save, still there after signing out and back in
- Keys kept out of the code and the repository, with a deny rule for `.env` files
- One Playwright test you watched fail before it passed
- `/security-review` run on a branch
- A GitHub repository with commits that show your progress
- You're ready to explain your work in class (below)

A one-shot submission, made by giving this brief to an agent once and turning in the result, is marked incomplete.

Be Ready to Explain

In class, you'll talk through your work. Be ready to explain:

1. **Tracing requests:** someone saves an item. What happens in the browser, on your server, in Clerk, in Supabase, and at the API?
2. **How you control access:** where does your app know who someone is, and where does it decide what they may see?
3. **Where API calls happen:** where your app calls the API, and why there.
4. **Testing:** what did your test look like when it failed, and what did you change?
5. **Agentic workflow:** what did you stop your agent from doing, and what did you catch when you read its changes?

Free Tiers and Usage Limits

Aside from Claude Code, you shouldn't need to pay for anything in this course. For every service you sign up for, such as Vercel, Supabase, Clerk, or a model API:

- Stay on the free tier.
- Set usage limits and spending alerts wherever the service offers them.
- Avoid adding a payment method unless a service requires one. If it does, set a spending cap first.

Free tiers have limits, too. If your app makes too many requests, a provider can slow it down, refuse requests for a while, pause your project, or block your key or account. Supabase pauses a free project after a week without use, so open your app once in a while until grades are back.

ASK YOUR AGENT

“Walk me through the free tier limits for each service my app uses, and how to set usage limits and spending alerts for each one.”